

PYTHON LEARNING PATH: FROM FOUNDATIONS TO ALGORITHMS (6 WEEKS)

WEEK 1: PYTHON BASICS - SYNTAX & OUTPUT

Starting guide to Python environment setup, mathematical operations, and data types.



Variable Assignment & Types

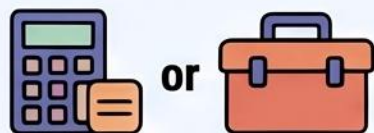
Variable Assignment & Types

Use `type()` to identify Integers, Floats, and Strings.



Arithmetic & Operators

Use `**` for exponents, `/` for division, and `//` for floor division.



Importing Math Functions

Use `from math import ...` or `import math` to access `sin`, `cos`, and `sqrt`.



Formatted Printing

Use f-strings (e.g., `f'{x:<15.2f}'`) to create clean, aligned data tables.

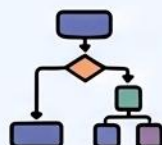
WEEK 2: LOGIC - LOOPS & FUNCTIONS

Moving from simple scripts to dynamic logic using loops and custom-defined functions.



For vs. While Loops

Use `for` for known iterations and `while` for conditions.



Imperative vs. Declarative

Imperative focuses on "how" (loops), Declarative on "what" (recursion).



Function Structure (def)

Functions use `return` to pass values back; procedures simply execute tasks.

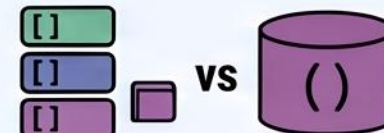


Lambda Expressions

Create quick, one-line anonymous functions using `lambda x: x**2`

WEEK 3: MANAGING DATA - LISTS TO DICTS

Efficiently organizing data using mutable and immutable collections.



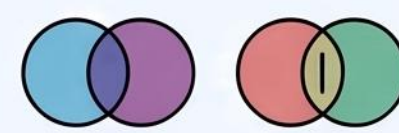
Lists vs. Tuples

Lists are mutable `[]`; Tuples are immutable `()` and faster.



Dictionaries for Mapping

Store data in Key-Value pairs for lightning-fast lookups.



Sets for Uniqueness

Sets `{}` automatically remove duplicates and allow set operations like Union.



Slicing & Indexing

Access sub-elements using `L[start:stop:step]` or `[::-1]` to reverse.

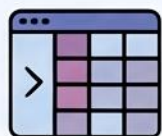
WEEK 4: DATA PROCESSING & I/O

Bridging the gap between code and external data sources like CSV and TXT files.



Reading CSV with Pandas

`pd.read_csv(filename)` instantly converts files into workable matrices.



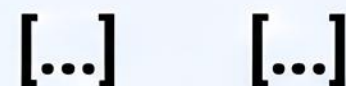
String Cleaning

Use `.replace()`, `.lower()`, and `.split()` to prepare text for analysis.



Frequency Counting

Use Dictionaries to count word occurrences in large texts efficiently.

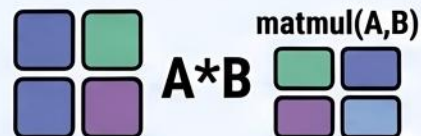


List Comprehensions

Create lists in one line: `[f(i) for i in range(N)]`

WEEK 5: SCIENTIFIC NUMPY: HIGH-PERFORMANCE MATH

Replacing standard lists with NumPy arrays for faster, multi-dimensional mathematical operations.



Element-wise vs. Matrix Math

`A * B` multiplies elements; `matmul(A, B)` performs linear algebra.



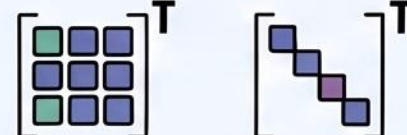
Array Slicing & Masks

Filter data using Boolean masks: `X[labels == 2, :]`



Matrix Allocation

Initialize data quickly with `zeros()`, `identity()`, or `array()`.



Trace and Transpose

Essential linear algebra tools included: `transpose(A)` and `trace(A)`.

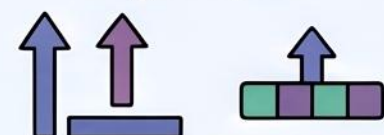
WEEK 6: ALGORITHMIC LOGIC - SOLVING LINEAR SYSTEMS

Applying Python logic to solve complex systems of linear equations manually.



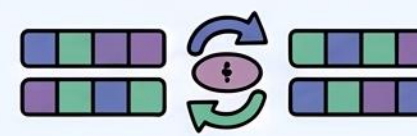
Forward Elimination

Systematically eliminate coefficients below the diagonal pivot.



Back Substitution

Solve for variables starting from the bottom row upwards.



Pivoting & Swapping

Swap rows to place the largest value on the diagonal to avoid division by zero.



Singular Matrices

If a pivot equals zero even after swapping, the system has no unique solution.